A semantic framework for symbolic execution in Maude

Rafael Morales-Palacios, Rubén Rubio, and Ignacio Fábregas

Universidad Complutense de Madrid, Madrid, Spain
{rafmor02, rubenrub, fabregas}@ucm.es

Abstract. Symbolic execution is a mainstream automated technique for analyzing computer programs. However, most implementations are developed for a specific programming language and without taking formal semantics into account. This paper introduces a general framework for specifying operational semantics in the Maude specification language, based on rewriting logic, and analyzing their programs using symbolic execution. User-defined semantics are translated to the appropriate setting for concrete execution, symbolic execution, or combinations of them, like concolic testing. Symbolic reasoning is mainly implemented through SMT solvers, but the possibilities of narrowing are also explored.

1 Introduction

Symbolic execution [4] is a widely-used approach for semiautomated program analysis, bug finding, test case generation, and sometimes functional verification. Its basic non-trivial idea is to run a program with symbolic values instead of actual inputs to cover the whole or a wider range of executions. Several techniques and tools have been developed to deal with specific programming languages, applications, and bug categories. Even the mainstream compilers for C and C++, GCC and Clang, include built-in analyzers based on symbolic execution. However, most of these tools are ad-hoc implementations not built on top of a formal definition of the language semantics, as already pointed out by the authors of $\mathbb{K}$ [27], a general framework for specifying formal semantics that is able to run symbolic executions for them. Some limitations of symbolic methods are the path explosion problem and the complexity of dealing with loops, complex or custom datatypes, memory, or the environment, which are not alien to other verification approaches. Hybridization with concrete executions, like concolic testing, can mitigate some of these problems.

Maude [9] is a high-level, high-performance specification and programming language based on rewriting logic [21]. It has been extensively used as a logic and semantic framework [20,31], where deduction and computation are naturally expressed by means of rules on a custom signature of terms modulo equations and structural axioms, like associativity and commutativity. Semantics for several languages have been specified and exploited in Maude [14,17,33]. Indeed, the already mentioned $\mathbb{K}$ framework was first developed using Maude. However, its

support for symbolic computation has substantially improved since then [12,22], with connections to SMT solvers [26] and symbolic reachability analysis via narrowing [11]. Moreover, the MaudeSE tool [34] has extended the built-in SMT support in Maude.

In this work, we develop a generic semantic infrastructure and some reflective transformations to run SMT-based symbolic executions on programs within Maude. Given a user-defined executable semantics satisfying some constraints, we derive a module where symbolic reachability problems can be solved by rewriting modulo SMT with MaudeSE or the Maude `search` command. Moreover, an extension of this transformation can be used for concolic testing, which involves simultaneous concrete and symbolic execution. Since concolic testing requires obtaining models from SMT formulae, which is not supported by Maude, we have developed an extension of the Maude SMT interface via the Maude Python library [28] for this purpose. Finally, a third translation shows that the framework can also be used for symbolic execution with narrowing. We illustrate the framework by detecting bugs and checking properties in some small examples written in a simple imperative language.

The paper is organized as follows: after discussing related work in Section 2 and some preliminaries in Section 3, the generic semantic infrastructure and its transformations are respectively presented in Section 4 and Section 5, and Section 6 illustrates it with some examples. Source code, these and more examples can be found in [23].

2 Related work

Some applications of symbolic execution are available in many end-user development tools. For instance, the widespread GNU compiler has recently incorporated a built-in bug finder based on coverage-guided symbolic execution for C programs; the Clang compiler already used it in its static analyzer for C, C++, and Objective-C; and Microsoft's Visual Studio is able to generate test cases by symbolic methods.[1] Other well-known open-source and commercial tools are KLEE [6], Facebook's Infer [7], GNAT SAS for Ada, or SonarQube. Some examples in Section 6 portray some of the typical simple checks available in these tools.

Symbolic execution tools are not usually based on a formal symbolic semantics of the underlying programming language, or this semantics is developed separately from its concrete counterpart. In [32], the authors propose a rule format to simultaneously specify both concrete and symbolic big-step operational semantics, and illustrate it with the specification of the typical *While* language. We use a similar *While* language in the examples of this paper, although with a small-step semantics. Other works have formalized symbolic execution in the context of denotational and operational semantics [18,5]. However, the closest reference for program analysis based on a given formal semantics is the $\mathbb{K}$ Framework [27], which actively maintains a symbolic execution engine implemented in

[1] See `gcc.gnu.org`, `clang-analyzer.llvm.org`, and `visualstudio.microsoft.com`.

Haskell. The theoretical foundations for symbolic execution in $\mathbb{K}$ and a first prototype in Java were developed by Lucanu, Rusu, and Arusoaie [19,3]. It relies on syntactic unification and the Z3 SMT solver for applying the semantic rules on program configurations with variables, and uses a program transformation to integrate this symbolic search into $\mathbb{K}$. This is similar to the approach we follow in Section 4. That tool was used for Hoare and reachability logic verification, and bounded LTL model checking with some examples, while concolic testing is mentioned as future work. At that time, $\mathbb{K}$ used Maude as a rewriting engine, so model checking was done by the Maude LTL model checker [13]. The support for symbolic execution parameterized on the semantics in $\mathbb{K}$ is much more developed than the work we present in this paper, but we are doing it using the general-purpose symbolic features of Maude and applying other methods like concolic testing and equational narrowing.

As mentioned in the introduction, symbolic reasoning features in Maude have experimented important advances in its latest versions [12,22], but they were available since a long time. The most significant application of narrowing is the Maude-NPA tool [16] for cryptographic protocol analysis, but we can also mention a symbolic model checker for quantum circuits [10]. Regarding the interaction with SMT solvers, networks of parameterized timed automata [2] and parametric time Petri nets [1] have recently been analyzed using the external extension MaudeSE [34]. However, we do not know about any attempt to apply the narrowing features and connections with SMT solvers in Maude to analyze programs parameterized on their language semantics.

3 Preliminaries

In this section, we recall some basic notions on rewriting logic and Maude [9], rewriting modulo SMT [26], symbolic execution, and concolic testing [30]. We assume the reader is familiar with order-sorted signatures Σ over a poset of sorts $(S, \leq)$, the free algebra $T_\Sigma(X)$ of terms over a signature Σ with variables X, substitutions $\sigma : X \to T_\Sigma(X)$, and their homomorphic extension to terms $\sigma : T_\Sigma(X) \to T_\Sigma(X)$, which, abusing notation, we denote by the same symbol.

Symbolic execution. Symbolic execution generalizes standard execution by operating on symbolic values representing arbitrary inputs rather than concrete data. In this context, every program state maintains a path condition ϕ, a first-order logic formula that accumulates constraints over the symbolic inputs. Upon encountering a conditional statement with guard ψ, the execution bifurcates into two paths, updating the respective path conditions with $\phi \wedge \psi$ and $\phi \wedge \neg \psi$. An execution state is deemed feasible if and only if its associated path condition is satisfiable, a verification step typically performed by an SMT solver.

Concolic testing. Concolic testing is a hybrid technique designed to address the limitations of pure symbolic execution, such as the path explosion problem and the handling of external system calls, by running both concrete and symbolic

executions at the same time. The process operates on a *dual state*, maintaining both concrete values and their symbolic counterparts. The execution begins with a concrete input that drives the program along a specific path. Simultaneously, the symbolic part collects the constraints satisfied by the concrete values to build a path condition ϕ. To systematically explore the program's state space and discover new paths, the algorithm selects a decision point along the current trace, negates its constraint, and asks an SMT solver for a model. The solver then generates a new concrete input that satisfies this modified condition, ensuring that the next execution deviates into an unexplored branch, thus combining the efficiency of concrete execution with the coverage of symbolic reasoning.

Rewriting logic and Maude. Maude [9] specifications correspond to rewrite theories $\mathcal{R} = (\Sigma, E \cup A, R)$ in rewriting logic [21]. We assume some knowledge on rewriting logic and Maude, but recall that terms are considered modulo some equations E and structural axioms A, and non-deterministic transformations on these terms are specified by rewrite rules R. Specifications are executable and terms can be reduced to normal form with the `reduce` command, and rewritten with rules with `rewrite`. In addition to matching and rewriting, Maude supports variant-based unification and narrowing modulo for terms with variables, as a quite general form of symbolic computation. We explain other details when they are required in the paper and refer to the Maude manual [9] for further information.

Rewriting modulo SMT. Rewriting modulo SMT combines term rewriting with SMT solving to allow symbolic reasoning on top of the first-order theories supported by SMT solvers, which usually include linear arithmetic on both integers and rationals. Given a quantifier-free formula ϕ over a set of variables X in a theory $\mathcal{T}$, we say φ is satisfiable if there is a model M within $\mathcal{T}$ (i.e., an assignment of constants to the variables in X compatible with all the axioms of $\mathcal{T}$) that satisfies φ. For its integration into term rewriting, we distinguish a *built-in subsignature* $\Sigma_0 \subseteq \Sigma$ in the order-sorted signature Σ, where $T_{\Sigma,s_0}(X_0) = T_{\Sigma_0,s_0}(X_0)$ for every sort $s_0 \in \Sigma_0$ and every set of built-in variables X_0. This means that all subterms of any built-in term are built-in, so that they can be interpreted in the associated theory $\mathcal{T}$ of the SMT solver. In this context, the system state is represented by *constrained terms* of the form (t, ϕ), where t is a term in $T_\Sigma(X)$ and ϕ is a Boolean formula over Σ_0 representing constraints on the variables in t. A rewrite rule $l \to r$ if ψ in $\mathcal{R}$ can be applied to a constrained term (t, ϕ) if there exists a substitution θ such that t matches $\theta(l)$ modulo A, and the accumulated constraint $\phi \wedge \theta(\psi)$ is satisfiable in $\mathcal{T}$. The result of such a transition is the new constrained term $(\theta(r), \phi \wedge \theta(\psi))$. Thus, execution paths correspond to sequences of satisfiable constraints, enabling symbolic reachability analysis. MaudeSE [34] supports rewriting modulo SMT with this kind of rules and keeps the accumulated constraints in its internal states.

$$\begin{array}{ccc} \mathcal{E} & \xrightarrow{\mathrm{eval}_{\mathcal{V}}} & \mathcal{V} \\ \downarrow & & \downarrow \\ Sem & \xrightarrow{\mathcal{U}_{\mathcal{V}}} & Sem_{\mathcal{V}} \end{array}$$

Fig. 1. General diagram of the transformations.

4 Infrastructure for the specification of semantics

In this section, we describe a simple infrastructure to specify small-step operational semantics in Maude, so that programs can be executed both concretely and symbolically as explained in Section 5. Figure 1 gives a general picture of the framework, where *Sem* represents the user-defined syntactic and semantic definition of the desired language. As usual [20], although on top of the provided infrastructure, the user should specify the following elements of the language semantics:

1. the syntax of the programming language as a protected extension of a predefined signature $\mathcal{E}$ of Boolean, integer, and rational expressions with constants, variables, and the usual arithmetic operators. These expressions should be interpreted as bare syntax that may be evaluated to different target types $\mathcal{V}$ as required;
2. the execution states of the semantics using a predefined specification of variable stores, i.e., mappings from variable names to values of the corresponding target type in $\mathcal{V}$;
3. and the semantic rules over these execution states, which may use the `eval` function provided by the framework to evaluate expressions to the target types. A typical rule has the form

   `crl` S `=>` S'[... `eval(`e'`,` s`)` ...] `if` `eval(`e`,` s`)` `=` `true` `.`

   where `eval` may be used with a Boolean expression in the condition, or in the right-hand side to set the values of the store. Unconditional rules and rules with other shapes are allowed too.

The cornerstone of the framework is the abstraction of the semantic values $\mathcal{V}$ of the basic expressions and their evaluation with `eval`. In the concrete execution setting, Boolean, integer, and rational expressions are evaluated to the predefined `Bool`, `Int`, and `Rat` sorts in Maude, so that programs can be executed as usual. However, they can also be evaluated to the sorts `Boolean`, `Integer`, and `Rat` of SMT formulae, as we do in Section 5.1; or to `BoolFVP` and `IntFVP` for narrowing in Section 5.3; among others.

Since semantic rules will need to be transformed reflectively, instead of a parameterized skeleton, the infrastructure is provided as a module `CONCRETE-FW` gathering the specifications of $\mathcal{E}$, $\mathcal{V}$, and $\mathrm{eval}_{\mathcal{V}}$ for concrete executions. The latter two are reflectively transformed for the different symbolic execution settings.

The signature $\mathcal{E}$ consists of three sorts `BExp`, `IExp`, and `RExp` for Boolean, integer, and real-valued expressions, respectively. Literals and variables of each type are wrapped with the `val` and x`v` constructors as

```
op val : Bool -> BExp [ctor] .   op bv : Qid -> BExp [ctor] .
op val : Int  -> IExp [ctor] .   op iv : Qid -> IExp [ctor] .
op val : Rat  -> RExp [ctor] .   op rv : Qid -> RExp [ctor] .
```

using the predefined Maude sort for constants, and the sort `Qid` of quoted identifiers for variable names. The signature of each type of expression is extended with the usual operators of the data type like conjunction, disjunction, order relations, arithmetic operations, etc.

The predefined variable stores come in the same three flavors, `BStore`, `IStore`, and `RStore`, and each is defined as a `Map`{`Qid`,s} from the variable names to the corresponding target sort s. Then, we combine all three in a structure `Stores`:

```
op _|_|_ : IStore RStore BStore -> Stores [ctor] .
```

Finally, an evaluation function is defined for each category of expressions, which take the generic form of

```
op eval : XExp Store -> XVal .
```

where X`Exp` is one of the expression sorts, *Store* is the store needed for evaluating X`Exp`, and *XVal* is its target value. These functions receive the variable stores, from which they obtain the value of the variables in the expression. In the concrete setting, and therefore in the predefined module, they are defined as

```
op eval : BExp Stores -> Bool .
op eval : IExp IStore -> Int .
op eval : RExp RStore -> Rat .
```

These functions are recursively defined on the syntax tree of the expressions. The `eval` function for `BExp` receives the full set of type `Stores` as Boolean expressions may contain integer or real-valued subexpressions. Those for `IExp` and `RExp` only receive the store they need, since they do not contain nested subexpressions of a different type. In Section 5, we show how `eval` functions are transformed to operate with symbolic data.

In the concrete setting, the definition of `eval` is straightforward: it unwraps the literals from the `val` constructor, obtain variable values from the store, and translate operators to their identical counterparts in the target sort. Later, we will discuss that, on the translations for symbolic and concolic execution, `eval` functions translate literals from concrete values to their SMT representation.

4.1 A simple *While* language

Let us illustrate the framework with the *While* language, a simple imperative programming language with assignments, conditionals, and loops. Its semantics is given as a system module `WHILE-MAUDE` that follows the requirements mentioned in the previous section, and extends the module `CONCRETE-FW` with the infrastructure of the concrete semantics. Instructions are terms of sort `Inst` representing the basic statements of the language, and programs are sequences of instructions of sort `Prog`. They are defined with the following operators:

```
mod WHILE-MAUDE is
  protecting CONCRETE-FW .
  sorts Inst Prog .
  subsort Inst < Prog .

  op skip : -> Inst [ctor] .
  op _;_  : Prog Prog -> Prog [ctor assoc id: skip] .
  op while_do{_} : BExp Prog -> Inst [ctor] .
  op if_then{_}else{_} : BExp Prog Prog -> Inst [ctor] .
  op if_then{_}        : BExp Prog -> Inst .

  var P : Prog . var B : BExp .
  eq if B then {P} = if B then {P} else {skip} .
```

In WHILE-MAUDE, we also define the constructor of an execution state as

```
  op <_|_> : Prog Stores -> State [ctor] .
```

and two additional functions, getStore and updateStore, to obtain and update the store of a given state, that are needed for the concolic transformation in Section 5.2.

```
  op getStore    : State -> Stores .
  op updateStore : State Stores -> State .
```

Semantic rules are represented by unconditional and conditional rewrite rules with conditions of the form $t = \text{true}$, where t is a Boolean term or the result of some evaluation function. For instance, the rules for the Boolean assignment, the conditional, and the while loop are the following:

```
  vars P P1 P2 : Prog . var STR : Stores .
  var  Q       : Qid  . var B   : BExp   .
   rl [assign-B] : < bv(Q) := B ; P | STR >
                 => < P | insert(Q, eval(B, STR), STR) > .

  crl [if-then] : < if B then {P1} else {P2} ; P | STR >
                => < P1 ; P | STR > if eval(B, STR) = true .

  crl [if-else] : < if B then {P1} else {P2} ; P | STR >
                => < P2 ; P | STR >
                if not eval(B, STR) = true .

  crl [while-loop] : < while B do { P1 } ; P | STR >
                   => < P1 while B do { P1 } ; P | STR >
                   if eval(B, STR) = true .

  crl [while-exit] : < while B do { P1 } ; P | STR >
                   => < P | STR >
                   if not eval(B, STR) = true .
  *** [...]
endm
```

Conditional rules may also include sort-membership t `:` s, matching t `:=` t', or rewrite t `=>` t' conditions, which are left unchanged by the translations. Hence, the `eval` functions should not occur on them.

4.2 Concrete program execution

Semantics specified as explained in the previous sections are already in the concrete setting and can be directly executed with the `rewrite` command, traced step by step with the `search` command, or even model checked with concrete inputs. Of course, the specification should satisfy the usual admissibility requirements [9] like confluence and termination of equations, coherence of rules, etc.

For example, using the *While* language in Section 4.1, consider we want to execute the following program P

```
if iv('x) < val(0) then { iv('y) := val(0); }
else { iv('y) := val(2) * iv('x); }
```

when `x` takes value 1. Using the state constructor, the initial state becomes $S =$ `<` P `| 'x |->I 1 | empty | empty >` and the final state of the program can be computed with `rewrite`:

```
Maude> rewrite S .
result State: < nil | 'x |->I 1, 'y |->I 2 | empty | empty >
```

More interesting execution examples are discussed in Section 6.

5 Translations for symbolic analysis

In order to make any semantics specified as explained in the previous section amenable for symbolic execution, we need to transform them so that computation is applied on symbolic semantic values instead of concrete ones. Different types of analyses and techniques require related but different transformations. This section presents three transformations (1) for direct symbolic execution by rewriting modulo SMT with the MaudeSE tool, (2) for concolic testing, and (3) for direct symbolic execution using narrowing. All these program transformations, but the last one, are reflectively implemented in Maude.

5.1 Symbolic execution with SMT

In Section 4, we explained that our framework works on a user-defined semantics *Sem* extending a signature $\mathcal{E}$ of logical and numerical expressions, and using some predefined variable stores. In the concrete execution setting, the semantic values $\mathcal{V}$ (i.e., the result of the expression evaluation functions, and the values bound to variables in the store) are the concrete predefined Maude types `Bool`, `Int`, and `Rat`. In this section, we present the SMT-based symbolic setting, where the semantic values are SMT formulae on the Maude SMT sorts `Boolean`, `Integer`, and `Real`, defined in the `smt.maude` file of the official distribution. For executing

the programs, standard rewriting is replaced by rewriting modulo SMT [26], as implemented by MaudeSE [34]. Hence, the initial values of the program variables or arguments can be symbolic expressions with variables instead of simple constants.

Once the variable stores keep symbolic values and the evaluation functions yield them, symbolic semantic rules are obtained by an almost identical transformation

```
crl S => S' if eval evalS(e, s) = (true).Bool (true).Boolean .
```

where `eval` is replaced by the symbolic evaluation function `evalS`, `true` is now of `Boolean` type, and S' may include evaluations of the `evalS` function to be saved in the variable store. In general, these rules may include other condition fragments, and `eval` may be nested inside expressions. These rules are ready for rewriting modulo SMT, as explained in Section 3, with MaudeSE.

As explained in Section 4, the user-defined semantics is already in the concrete setting, and we translate it to a different setting using a reflective module transformation. This transformation consists of (1) including the predefined SMT modules and some useful extensions for converting the literals in the expressions to SMT constants, (2) replacing variable stores to concrete values with variable stores to symbolic values, (3) adapting the signature of the `eval` functions to yield SMT types instead of the standard ones, use symbolic stores, and perform the conversion of literals. For convenience, we use different names for sorts like `BStoreS`, `IStoreS`, and `RStoreS`, and evaluation functions, since symbolic execution will have to coexist with concrete execution in Section 5.2. The signature of the transformed evaluation functions is

```
op evalS : BExp Stores -> Boolean .
op evalS : IExp IStoreS -> Integer .
op evalS : RExp RStoreS -> Real .
```

and variable stores are now `Map{Qid,Boolean}` instead of `Map{Qid,Bool}`, and so on.

We can run programs under the symbolic semantics using the `metaSmtSearch` function from MaudeSE, which is the metalevel counterpart of its `smt-search` command. It searches through all SMT-rewriting paths, like the `search` command does over the standard rewriting paths. While MaudeSE provides significant advantages with respect to the built-in SMT support, like folding and extensibility, we can alternatively implement rewriting modulo SMT in Maude itself. This can be achieved by extending the state with an accumulated path constraint

```
op _{_} : State Boolean -> SEState [ctor] .
```

that is extended by the semantic rules as MaudeSE does under the hood:

```
crl S {C} => S' {C'}
  if C' := C and evalS(e, STR)
  /\ metaCheck(['REAL-INTEGER], upTerm(C')) .
```

where `metaCheck` is used to check whether the constraints are satisfiable using the SMT solver on the real and integer arithmetic theory in the predefined `REAL-INTEGER` module. Operator `op [_] : Qid → Module` generates a new module that imports the target one efficiently. Moreover, `metaCheck` takes the metarepresentation of the SMT formula representing the constraints, and we use the `upTerm` meta-level function to obtain it from its object level term. As an example, the rule that takes the positive branch of a conditional will extend the constraint with the evaluation of its condition and will be applied only when this extended constraint is satisfiable, i.e., when taking the positive branch is possible. This explicit translation will be used for concolic testing in Section 5.2.

We can symbolically execute program P from Section 4.2 with a Maude search over the transformed module. After one step, we have two states, one for each branch of the conditional.

```
search < P | ('x |->Is x:Integer | empty | empty) > {true}
    =>1 < P':Program | STR:Stores > {CST:Boolean} .

Solution 1 (state 1)
STR:Stores --> 'x |->Is x:Integer | empty | empty
CST:Boolean --> true and x:Integer < 0
P':Program --> iv('y) := val(0) ;

Solution 2 (state 2)
STR:Stores --> 'x |->Is x:Integer | empty | empty
CST:Boolean --> true and not x:Integer < 0
P':Program --> iv('y) := val(2) * iv('x) ;
```

Once the transformed conditional rule is applied, each state has accumulated the corresponding path constraint, as expected.

In order for symbolic execution to be useful, it must soundly and/or completely simulate concrete executions. Let Σ_0 be the built-in subsignature of the SMT theory of linear integer and real arithmetic in Maude; recall from Section 3 that X_0 denotes the set of built-in variables; $\mathcal{L}$ the subsignature of standard Boolean, integer, and rational constants in Maude; $\mathrm{Var}(t)$ denote the set of variables in t; and $\mathcal{U}$ the symbolic transformation (overloaded for modules, terms, rules, etc.). The implementation details on $\mathcal{U}$ can be found in the code repository.

Lemma 1. *For any concrete store $\sigma_c : V \to \mathcal{L}$, symbolic store $\sigma_s : V \to T_{\Sigma_0}(X_0)$, and expression $e \in T_{\mathcal{E}}(V)$, $\mathrm{Var}(\mathrm{eval}_s(e, \sigma_s)) \subseteq \bigcup_{x \in V} \mathrm{Var}(\sigma_s(x))$ and for every $\theta : X_0 \to T_{\Sigma_0}(\emptyset)$ such that $\mathcal{U}(\sigma_c(x)) =_{\mathcal{T}} \theta(\sigma_s(x))$ for all $x \in V$, $\mathcal{U}(\mathrm{eval}_c(e, \sigma_c)) =_{\mathcal{T}} \mathrm{eval}_s(e, \sigma_s)$.*

Definition 1. *A concrete state c and a symbolic state (s, ϕ) are* paired *if c and s only differ in their stores, respectively, σ_c and σ_s, $\mathrm{dom}(\sigma_c) = \mathrm{dom}(\sigma_s)$, and there is a substitution $\theta : X_0 \to T_{\Sigma_0}(\emptyset)$ such that $\mathcal{T} \models \theta(\phi)$ and $\mathcal{U}(\sigma_c(x)) =_{\mathcal{T}} \theta(\sigma_s(x))$ for all $x \in \mathrm{dom}(\sigma_c)$.*

Notice that (s, ϕ) is satisfiable iff it is paired with some concrete state.

Proposition 1. *1. For any concrete states c and c' such that $c \to_R c'$, and any symbolic state (s, ϕ) paired with c, there is a symbolic state (s', ϕ') paired with c' such that $(s, \phi) \to_{\mathcal{U}(R)} (s', \phi')$.*

2. For any symbolic states (s, ϕ) and (s', ϕ') such that $(s, \phi) \to_{\mathcal{U}(R)} (s', \phi')$, and any concrete state c' paired with (s', ϕ'), there is a concrete state c paired with (s, ϕ) such that $c \to_R c'$.

Corollary 1. *Symbolic execution on $\mathcal{U}(Sem)$ is sound and complete, i.e.,*

- *For any symbolic run $(s_1, \phi_1) \to_{\mathcal{U}(R)} \cdots \to_{\mathcal{U}(R)} (s_n, \phi_n)$ where ϕ_n is satisfiable, there is a concrete run $c_1 \to \cdots \to c_n$ where c_k is paired with (s_k, ϕ_k) for all $1 \leq k \leq n$.*
- *For any concrete run $c_1 \to \cdots \to c_n$, there is a symbolic run $(s_1, \phi_1) \to_{\mathcal{U}(R)} \cdots \to_{\mathcal{U}(R)} (s_n, \phi_n)$ where c_k is paired with (s_k, ϕ_k) for all $1 \leq k \leq n$.*

5.2 Concolic testing

Symbolic execution suffers from the path explosion problem, especially when loops or recursion are involved, and cannot be easily applied to complex programs. Concolic testing addresses this problem by combining repeated symbolic and concrete executions, as explained in Section 3. Moreover, this procedure requires a custom interaction with the SMT solver that cannot be achieved with the `smt-search` command of MaudeSE. Hence, our implementation of concolic testing deals with rewriting modulo SMT in Maude itself as explained at the end of the previous section. Our execution states are now of the form

```
op [_][_{_}][_] : State State Boolean State
                  -> ConcolicState [ctor].
```

where *State* is the state sort defined in the semantics. The first and second arguments are respectively a concrete and symbolic execution state with the accumulated path constraints inside braces. These arguments will be paired (as in Definition 1) during concolic executions. Finally, the fourth argument contains a copy of the initial symbolic state to be used when the program is restarted.

Unconditional `rl` S `=>` S' and conditional `crl` S `=>` S' `if` C rules in the user specification are translated to rules of the form

```
vars CST CST' : Boolean . var SMSInit : State .
 rl [S][U(S) {CST}][SMSInit] => [S'][U(S') {CST}][SMSInit] .
crl [S][U(S) {CST}][SMSInit] => [S'][U(S') {CST'}][SMSInit]
 if C /\ CST' := CST and U(C) .
```

where conditional rules check the concrete condition C and accumulate its symbolic translation to the path constraint. Notice that we do not have to check the satisfiability of `CST'` because the symbolic state is paired with the concrete one. For example, the rules for the Boolean assignment and the positive branch of conditional of the *While* language become

```
rl [assign-B] : [< bv(Q) := B ; P | STR >]
   [< bv(Q) := B ; P | STRS > {CST}][SMSInit]
=> [< P | insert(Q, eval(B, STR), STR) >]
   [< P | insert(Q, evalS(B, STRS), STRS) {CST}][SMSInit] .

crl [if-then] : [< if B then {P1} else {P2} ; P | STR >]
    [< if B then {P1} else {P2} ; P | STRS > {CST}][SMSInit]
 => [< P1 P | STR >][< P1 P | STRS > {CST'}][SMSInit]
 if eval(B, STR) = true /\ CST' := CST and evalS(B, STRS) .
```

where actions are independently replicated in both states.

While these new rules on the extended states allow running paired concrete and symbolic executions, it remains to implement the control logic of the concolic testing. Given an initial concrete state S, an initial symbolic store s_0 on the inputs of the program, and an initial constraint ϕ_0, the variant of concolic testing we consider proceeds as follows:

1. Generate concrete values for the inputs of the program using s_0 and ϕ_0, and build an initial concrete variable store s with them. If the stores in S already contain initial (concrete) assignments, they are kept, and the store is completed with assignments satisfying ϕ_0 for the remaining variables. The SMT solver can be used to generate a model for ϕ_0 as we will explain later in this section.
2. Build an initial concolic state `[`S'`][`O $\{\phi_0\}$`][`O`]` where $S' =$ `updateStore(` S`,` s`)` and $O =$ `updateStore(`S`,` s_0`)` using the user-defined `updateStore` function.
3. Run the concolic state with the transformed rules until no more rewrites are possible or a depth bound is reached. Moreover, when a conditional semantic rule is encountered, if its concrete condition is not satisfied, check whether the path constraint extended with the symbolic condition of the rule is satisfied. If so, obtain a model for that constraint ϕ, build an initial concrete variable store for it as in (1) (with ϕ instead of ϕ_0), and restart the execution from the beginning. The initial symbolic state is recovered from the last argument of the concolic state, and the corresponding concrete state is generated from it.

Notice that the second part of the third step spawns a new execution path, but the current execution can still continue with other semantic rules that succeed in the concolic state. Since every feasible road is taken in every execution crossroad, the concolic procedure is expected to test a wide and significant range of behaviors. Then, properties can be checked on the execution paths or the visited states.

The procedure above can be implemented as a standard Maude search on the transformed module if we add some additional rules. For each conditional semantic rule `crl` S `=>` S' `if` C, we generate a second transformed rewrite rule

```
crl [S][U(S) {CST}][SMSInit]
 => [S'][SMSInit {CST'}][SMSInit]
```

```
if not C /\ CST' := CST and not U(C)
/\ metaCheck(['REAL-INTEGER], upTerm(CST'))
/\ S' := updateStore(SMSInit,
                     initSTR(getStore(SMSInit), CST')) .
```

where `S'` is the new initial concrete state with the inputs obtained from the SMT solver as a model of `CST'`. For example, in the positive conditional rule, `if-then`, S would be `< if B then {P1} else {P2} ; P | STR >`, $\mathcal{U}(S)$ would be the same but with `STRS` instead of `STR`, C would be `eval(B, STR)`, and $\mathcal{U}(C)$ would be `evalS(B, STRS)`. Observe that we are checking that `CST'` is satisfiable with the `metaCheck` metalevel function of the official distribution. Moreover, `initSTR` needs to obtain a model for `CST'` in order to initialize the concrete variables of `S'`, but the Maude interface to the SMT solver only returns a Boolean answer. We have solved this problem by implementing a custom hook that communicates directly to the Z3 [24] SMT solver and returns a substitution:

```
op get-SMTassignment : Boolean -> AssignmentSMT
    [special (id-hook SpecialHubSymbol)] .
```

where `AssignmentSMT` is a comma-separated list of assignments `_<--_` whose arguments are either `Boolean`, `Integer`, or `Real`. This hook is implemented in Python using the Maude Python library [28]. Notice that the path constraint `CST'` is checked twice by an SMT in the transformed rule, first by the `metaCheck` function and then by `get-SMTassignment`, if it is satisfiable. Even though the invocation of `metaCheck` is redundant, it appears to reduce the execution time in the examples in Section 6. This might be caused by the more direct and efficient integration of `metaCheck` in Maude with respect to the Python-based hook, or the additional cost of generating a model.

In summary, the user-defined semantics *Sem* are translated to a version amenable for concolic testing Conc(*Sem*) by (1) including the infrastructure for both concrete and symbolic execution of the previous translations, with bidirectional conversions of constants between regular and SMT types, (2) duplicating the `eval` functions and the variable stores, and transforming the definition of symbolic copy as in Section 5.1, (3) defining a concolic state from the original state definition, (4) translating the original rules to advance the concolic execution state, (5) adding alternative rules for restarting the execution with different concrete inputs when a condition semantic rule cannot be applied, and (6) implementing a hook and some auxiliary definitions (both in Maude and Python) to obtain models from SMT formulae, as required by the method.

Since Conc(*Sem*) is produced at the metalevel, we are forced to interact with it from the metalevel too. Moreover, since it uses a Python-implemented hook, it needs to be run through the Maude Python library. In order to overcome these problems, which may appear when using other reflective transformations, we have developed `maude-shell` [29], a prototype of an extended Maude interpreter inspired in Full Maude [8], but implemented in Python. This tool allows loading Python hooks and selecting metamodules with the `select` command to execute regular commands on them, although not all commands are currently supported.

For instance, to check whether a property P is satisfied by concolic testing, we run

```
$ maude-shell maudeSMTHook.py \
              while.maude semantics-analysis-tr.maude
Maude> select concTr(upModule('WHILE-MAUDE, true), 'State)
Maude> search initial =>* CS s.t. not P .
```

and check there are no solutions. As claimed before, we have reduced concolic testing to a Maude search, where only the hook `get-SMTassignment` is implemented outside Maude. This allows using all command and features of Maude on the transformed model, like bounding the depth of the search, inspecting the rewrite graph, etc.

5.3 Symbolic execution via narrowing

In addition to SMT, Maude supports symbolic computation through variant-based equational unification and narrowing on user-defined theories satisfying the *finite variant property* [11]. This class of theories is more general than the repertory available in SMT solvers, although the general procedure would not be as efficient as the specialized decision procedures of these tools. We show here, as a proof of concept, how to adapt our framework for symbolic reasoning with narrowing on specifications using as semantic values the sorts `BoolFVP` of the Booleans and `IntFVP` of the Presburger integer arithmetic, which we have taken or extended from [15]. Both satisfy the finite variant property. As in previous transformations, we modify `eval` to produce terms in the `BoolFVP` and `IntFVP` sorts, assuming that rationals and unsupported operators (like product) are absent from the programs. However, since narrowing only works on top and for unconditional rules in Maude, the translation of semantic rules is more involved and produces for every conditional rule `crl` S `=>` S' `if eval(`e`, STR) = `b two narrowing-enabled rules using a temporary extended state

```
rl S => < S | eval(e, STR) > [narrowing] .
rl < S | b' > => S' [narrowing] .
```

where b' is either `tt` or `ff`, the Boolean constants in `BoolFVP`, if b was respectively `true` or `false`. This way, the Boolean formula that results from `eval(`e`, STR)` is unified against `tt` or `ff` modulo the variant equations in the specification and the state is narrowed accordingly. Then, one can symbolically execute programs using narrowing with the `vu-narrow` command, as we show in Section 6.1.

6 Case studies

In this section, we illustrate the proposed transformation and its associated techniques with some simplified examples. Although the examples here are defined with the *While* language specified in Section 4.1, tests with a simple functional recursive language can be found in the repository [23].

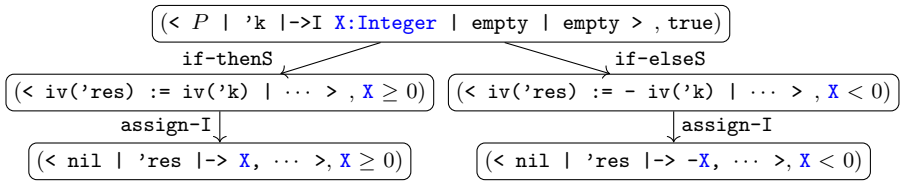

Fig. 2. Graph of a symbolic execution of P_1.

6.1 Checking functional properties

Consider a program P_1 in the *While* language of Section 4.1 that calculates the absolute value of an input variable `k` as

```
if iv('k) ≥ val(0) then { iv('res) := iv('k); }
else { iv('res) := - iv('k); }
```

In the concrete setting, we can fix the initial value of `k` and execute the program with the `rewrite` command or obtain an execution trace with `search`. For instance, if `k` takes value 0, and assuming arbitrary real-valued and Boolean stores `RS` and `BS`, the execution will follow this path

```
Maude> search < P1 | 'k |->I 0 | RS | BS > =>! S:State .
...
Maude> show search graph .
state 0, State: < P1 | 'k |->I 0 | RS | BS >
arc 0 ===> state 1 crl ... [if-then]
state 1: < iv('res) := iv('k) ; | 'k |->I 0 | RS | BS >
arc 0 ===> state 2 rl ... [assign-I]
state 2: < nil | ('k |->I 0, 'res |->I 0) | RS | BS >
```

For this small example, its behavior can be easily characterized by the symbolic execution graph in Figure 2, where the value of `res` can be either `X` or `-X` at the end of the program, where `X` is the initial value of `k`. This is the SMT search graph in the transformed module $\mathcal{U}(Sem)$ presented in Section 5.1. The `semantics-analysis-ext.py` script available in the companion repository [23] facilitates loading a given semantics, transforming it for the desired analysis, and executing commands on it. For instance, we can check that `res` is always positive at the end of P_1 with a command equivalent to

```
> smt-search < P1 | 'k |->I X:Integer | RS | BS >
          =>! < nil | 'res |->I Y:Integer, IS | RS | BS >
          s.t. not (Y:Integer ≥ 0) .
```

that will have no solution, and it takes around 180ms.

Alternatively, we can check that the final value of `res` is always positive by using narrowing with the procedure in Section 5.3. Unlike the other transformations, this is not implemented in Maude, so we have translated the module by hand. Since the property holds, if we look for a final state where `res` is bound to a negative value (represented by the pattern `- Y:NzNatFVP`) from an initial state where `k` takes an arbitrary integer value `X:IntFVP`, we must find no solution.

```
Maude> vu-narrow {delay, filter} < P1 | 'k |->I X:IntFVP >
          =>* < skip | 'res |->I - Y:NzNatFVP, IS > .
No solution.
rewrites: 147 in 3ms cpu (3ms real) (49000 rewrites/second)
```

Moreover, if we replace `- Y:NzNatFVP` with `Y:NzNatFVP` in the pattern, i.e., a negative result with a positive one, the same query provides two solutions where `X` respectively equals `Y` and `- Y`, as expected. In other words, we have calculated the preimages of the absolute value under different constraints, and this can also be applied to other functions.

6.2 Finding bugs by symbolic execution

We can use symbolic execution to find bugs or undesired situations in programs, for example, a division by 0. Consider a program P_2 where the denominator of a division may be 0 at some point in execution. P_2 is defined as

```
rv('y) := val(20); rv('i) := val(5);
while rv('k) < rv('y) do {
  if rv('k) > val(10) then { rv('z) := rv('y) / rv('i); }
  rv('k) := rv('k) + val(1);
  rv('i) := rv('i) - val(1); }
```

where `k` is a symbolic rational variable. We can execute this program symbolically and look for a state matching the pattern `< RV:RVar := RE:RExp / rv('i ); P:Prog | IS:IStoreS | RS:RStoreS, 'i |->Rs 0/1 | BS:BStoreS >` where the denominator variable is zero. The result of the SMT-search, which took around twice the time as for P_1, indicates that the state number 25 matches the pattern, and it is reachable when $10 < \mathtt{k} < 20$ after 5 iterations of the loop.

6.3 Finding bugs in more complex programs by concolic testing

As we have already mentioned, direct symbolic execution may become unfeasible when the program becomes more complex due to path explosion. In a program like the following

```
iv('i) := val(5);
while iv('k) < iv('y) do {
  if iv('k) > val(10) then {
    iv('x) := iv('k) * iv('y);
    iv('a) := iv('x) - iv('i);
    iv('z) := iv('y) / iv('a); }
  iv('k) := iv('k) + val(1); iv('i) := iv('i) * val(2); }
```

where `k` and `y` are symbolic variables, the conditional inside the loop forks the path in each iteration, making the symbolic graph grow exponentially. With concolic execution, we can mitigate this problem, as it executes the program concretely several times. Similarly to the previous examples, using the transformed module, we can search over the concolic state graph for a state where

we are dividing by some variable with value 0 at that execution point. This can be done with the extended Maude interpreter presented in Section 5.2 and the standard `search` command. Maude finds a solution at state 3147 after around 1.3 seconds. We can retrieve the sequence of rules applied with the `show path` command, and check that the bug was reached after 7 iterations of the loop. The resulting store shows that the counterexample corresponds to the assignment: $k \mapsto 9$, $y \mapsto 20$.

7 Conclusions

In this paper, we have presented a generic framework for symbolic and concolic execution of programs parameterized on the semantics of the language in Maude. This work is built on top of the existing support for symbolic computation in Maude and MaudeSE, using mainly rewriting modulo SMT but also narrowing. We have implemented metalevel translations from a user-defined semantics with some restrictions to modules ready for symbolic and concolic execution. For the latter, we have also implemented a concolic execution algorithm with the transformed semantic rules, aided by a custom Maude hook in Python that generate initial values for concrete execution of the algorithm via SMT. In addition, we provide auxiliary Python scripts to simplify using the framework, and a general prototype inspired in Full Maude to run object-level commands on metalevel modules with potential Python hooks [29]. These translations are generic for any semantic specification, and have been tested with a simple imperative language and a functional recursive language. Examples of both can be found in the code repository [23]. Finally, we run a small benchmark the different symbolic execution approaches.

As future work, we consider incorporating narrowing and narrowing modulo SMT as a way of dealing with more complex data types and custom data structures, as well as trying symbolic model checking to verify more complex properties. We also want to apply the framework on semantics for more realistic programming languages, like WebAssembly [25].

Acknowledgments. We would like to thank Narciso Martí-Oliet for his comments on this manuscript. This work is partially supported by MCIU/AEI/10.13039/50110-0011033/FEDER,UE through project ProCode 10 (PID2023-149943OB-I00). Rubén Rubio is partially supported by Comunidad de Madrid/UCM through project COMFIA (PR17/24-31913).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the contents of this article.

References

1. Arias, J., Bae, K., Olarte, C., Ölveczky, P.C., Petrucci, L.: A rewriting-logic-with-SMT-based formal analysis and parameter synthesis framework for parametric time Petri nets. Fundam. Informaticae **192**(3-4), 261–312 (2024). https://doi.org/10.3233/FI-242195

2. Arias, J., Bae, K., Olarte, C., Ölveczky, P.C., Petrucci, L., Rømming, F.: Symbolic analysis and parameter synthesis for networks of parametric timed automata with global variables using Maude and SMT solving. Sci. Comput. Program. **233**, 103074 (2024). https://doi.org/10.1016/j.scico.2023.103074
3. Arusoaie, A.: A Generic Framework for Symbolic Execution: Theory and Applications. Ph.D. thesis, Alexandru Ioan Cuza University, Iaşi, Romania (2014), https://tel.archives-ouvertes.fr/tel-01094765
4. Baldoni, R., Coppa, E., D'Elia, D.C., Demetrescu, C., Finocchi, I.: A survey of symbolic execution techniques. ACM Comput. Surv. **51**(3), 50:1–50:39 (2018). https://doi.org/10.1145/3182657
5. de Boer, F.S., Bonsangue, M.M.: Symbolic execution formally explained. Formal Aspects Comput. **33**(4-5), 617–636 (2021). https://doi.org/10.1007/S00165-020-00527-Y, https://doi.org/10.1007/s00165-020-00527-y
6. Cadar, C., Nowack, M.: KLEE symbolic execution engine in 2019. Int. J. Softw. Tools Technol. Transf. **23**(6), 867–870 (2021). https://doi.org/10.1007/s10009-020-00570-3
7. Calcagno, C., Distefano, D.: Infer: An automatic program verifier for memory safety of C programs. In: NFM 2011. LNCS, vol. 6617, pp. 459–465. Springer (2011). https://doi.org/10.1007/978-3-642-20398-5_33
8. Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Talcott, C.L.: Full Maude: Extending Core Maude. In: All About Maude - A High-Performance Logical Framework, How to Specify, Program and Verify Systems in Rewriting Logic, LNCS, vol. 4350, pp. 559–597. Springer (2007). https://doi.org/10.1007/978-3-540-71999-1_18
9. Clavel, M., Durán, F., Eker, S., Escobar, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.: Maude Manual (version 3.5.1) (2025), https://maude.lcc.uma.es/maude-manual/
10. Do, C.M., Ogata, K.: Symbolic model checking quantum circuits in Maude. PeerJ Comput. Sci. **10**, e2098 (2024). https://doi.org/10.7717/peerj-cs.2098
11. Durán, F., Eker, S., Escobar, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.L.: Equational unification and matching, and symbolic reachability analysis in Maude 3.2 (system description). In: IJCAR 2022. LNCS, vol. 13385, pp. 529–540. Springer (2022). https://doi.org/10.1007/978-3-031-10769-6_31
12. Durán, F., Eker, S., Escobar, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.L.: Programming and symbolic computation in Maude. J. Log. Algebraic Methods Program. **110** (2020). https://doi.org/10.1016/j.jlamp.2019.100497
13. Eker, S., Meseguer, J., Sridharanarayanan, A.: The Maude LTL model checker and its implementation. In: SPIN 2003. LNCS, vol. 2648, pp. 230–234. Springer (2003). https://doi.org/10.1007/3-540-44829-2_16
14. Ellison, C., Rosu, G.: An executable formal semantics of C with applications. In: Field, J., Hicks, M. (eds.) POPL 2012. pp. 533–544. ACM (2012). https://doi.org/10.1145/2103656.2103719
15. Escobar, S.: Extensions of logic programming in Maude (2023), https://logicprogramming.org/2023/02/extensions-of-logic-programming-in-maude/
16. Escobar, S., Meadows, C., Meseguer, J.: Maude-NPA: Cryptographic protocol analysis modulo equational properties. In: FOSAD 2007/2008/2009. LNCS, vol. 5705, pp. 1–50. Springer (2007). https://doi.org/10.1007/978-3-642-03829-7_1
17. Farzan, A., Chen, F., Meseguer, J., Rosu, G.: Formal analysis of Java programs in JavaFAN. In: Alur, R., Peled, D.A. (eds.) CAV 2004. LNCS, vol. 3114, pp. 501–505. Springer (2004). https://doi.org/10.1007/978-3-540-27813-9_46

18. Kneuper, R.: Symbolic execution: A semantic approach. Sci. Comput. Program. **16**(3), 207–249 (1991). https://doi.org/10.1016/0167-6423(91)90008-L
19. Lucanu, D., Rusu, V., Arusoaie, A.: A generic framework for symbolic execution: A coinductive approach. J. Symb. Comput. **80**, 125–163 (2017). https://doi.org/10.1016/j.jsc.2016.07.012
20. Martí-Oliet, N., Meseguer, J.: Rewriting logic as a logical and semantic framework. In: Meseguer, J. (ed.) RWLW 1996. ENTCS, vol. 4, pp. 190–225. Elsevier (1996). https://doi.org/10.1016/S1571-0661(04)00040-4
21. Meseguer, J.: Conditional rewriting logic as a unified model of concurrency. Theoretical Computer Science **96**(1), 73–155 (1992). https://doi.org/10.1016/0304-3975(92)90182-F
22. Meseguer, J.: Symbolic computation in Maude: Some tapas. In: Fernández, M. (ed.) LOPSTR 2020. LNCS, vol. 12561, pp. 3–36. Springer (2020). https://doi.org/10.1007/978-3-030-68446-4_1
23. Morales, R.: Symbolic execution in Maude (2026), https://github.com/rafamor-exe/Symbolic_Execution_in_Maude
24. de Moura, L.M., Björner, N.S.: Z3: an efficient SMT solver. In: TACAS 2008. LNCS, vol. 4963, pp. 337–340. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24
25. Palacios, R.M.: Semantics of WebAssembly in Maude. Master's thesis, Facultad de Informática. Universidad Complutense de Madrid (2025), https://hdl.handle.net/20.500.14352/125016
26. Rocha, C., Meseguer, J., Muñoz, C.A.: Rewriting modulo SMT and open system analysis. J. Log. Algebraic Methods Program. **86**(1), 269–297 (2017). https://doi.org/10.1016/j.jlamp.2016.10.001
27. Rosu, G., Serbanuta, T.: An overview of the K semantic framework. J. Log. Algebraic Methods Program. **79**(6), 397–434 (2010). https://doi.org/10.1016/j.jlap.2010.03.012
28. Rubio, R.: Maude as a library: An efficient all-purpose programming interface. In: WRLA 2022. LNCS, vol. 13252, pp. 274–294. Springer (2022). https://doi.org/10.1007/978-3-031-12441-9_14
29. Rubio, R.: Maude Shell: a extended Maude interpreter prototype (2026), https://github.com/fadoss/maude-shell
30. Steinhöfel, D.: Symbolic Execution: Foundations, Techniques, Applications, and Future Perspectives, pp. 446–480. Springer (2022). https://doi.org/10.1007/978-3-031-08166-8_22
31. Verdejo, A., Martí-Oliet, N.: Executable structural operational semantics in Maude. J. Log. Algebraic Methods Program. **67**(1-2), 226–293 (2006). https://doi.org/10.1016/j.jlap.2005.09.008
32. Voogd, E., Johnsen, E.B., Kløvstad, Å.A.A., Rot, J., Silva, A.: Correct and complete symbolic execution for free. In: IFM 2024. LNCS, vol. 15234, pp. 237–255. Springer (2024). https://doi.org/10.1007/978-3-031-76554-4_13
33. Xiao, L., Zhu, H.: UTP semantics for the MCA ARMv8 architecture. J. Syst. Archit. **125**, 102438 (2022). https://doi.org/10.1016/j.sysarc.2022.102438
34. Yu, G., Bae, K.: A flexible framework for integrating Maude and SMT solvers using Python. In: WRLA 2024. LNCS, vol. 14953, pp. 179–192. Springer (2024). https://doi.org/10.1007/978-3-031-65941-6_10